End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning (DM4ML) - Assignment 2 · BITS Pilani WILP, M.Tech AI/ML

Soma Sekhar Reddy Vattakula · 2025AG05032 · 12 November 2026

GitHub repository: https://github.com/vssr32000/recommendation-pipeline

1. Problem and approach

The task is to recommend movies to users of an online movie platform from their past ratings and item metadata, and, more importantly, to build the **data management pipeline** around the model so that it is validated, reproducible and automated. I used the public **MovieLens-1M** dataset (6,040 users with demographics, 3,883 movies with genres, 1,000,209 timestamped 1-5 star ratings). The pipeline has seven stages, each a separate Python module with a `run(cfg)` entry point, driven by one YAML config and orchestrated as a Prefect flow (Figure 1).

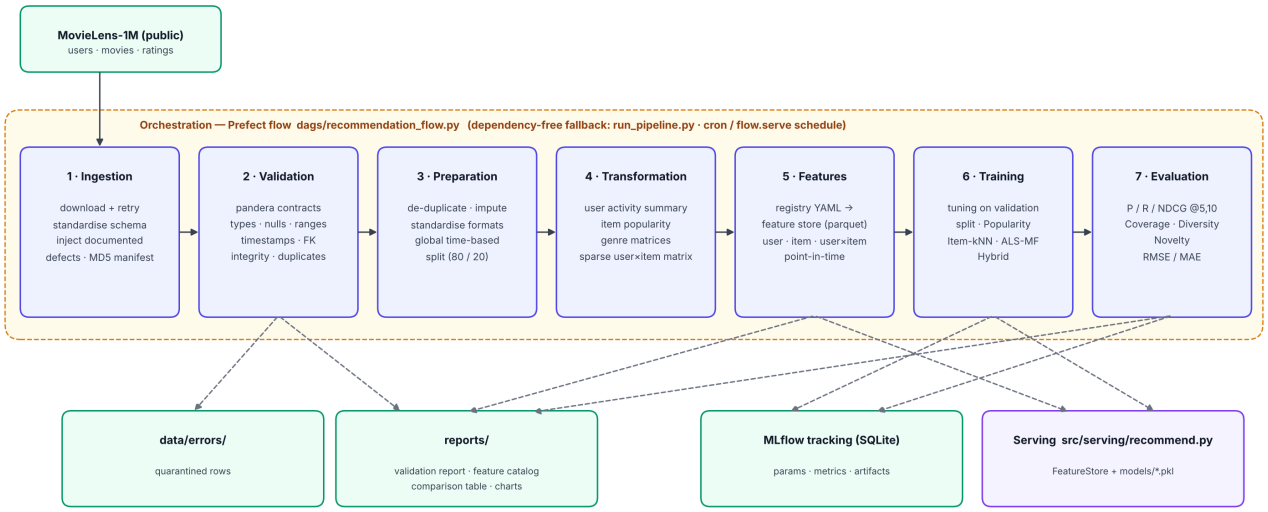


Figure 1 - Pipeline architecture. Solid arrows: task dependencies; dashed arrows: outputs.

2. Design decisions and tools

Stage	Tool(s)	Key design decision
Ingestion	urllib, pandas	Download with retries and a local cache; standardise all sources into one users / items / interactions contract; MD5 + row-count manifest for lineage. Because MovieLens is already clean, 880 seeded, logged defects are injected so validation can be tested against a known answer.
Validation	pandera	Declarative schemas per table. Every rule has a severity: *error* rows (wrong type, null key, invalid rating/timestamp, orphan FK) are quarantined with reasons; *warning* rows (duplicates, missing optional attributes) pass and are repaired downstream, so no user is dropped and foreign keys stay valid. A data-quality gate fails the run above 1% rejected rows.
Preparation	pandas, parquet	Deterministic cleaning (latest re-rating wins, mode/'U'/'Unknown' imputation, UTC datetimes, parsed release year). One **global time cut-off** (80th percentile) so no future rating can leak into training, which per-user or random splits allow.
Transformation	pandas, scipy.sparse	Derived datasets come from the train split only: user activity summary, item popularity with a Bayesian-smoothed rating, genre matrices, and a CSR user-item matrix.
Features	Custom feature store (YAML registry + parquet)	One registry defines each feature; the build fails if a declared feature is missing; the catalog is generated from the registry. The same `FeatureStore` API serves training, evaluation and the serving CLI.
Training	NumPy/SciPy, MLflow	Four models behind one interface. Hyper-parameters are tuned on the last 10% of the *train* period (never the test set). MLflow logs every trial as a nested run plus params, lineage tags (cut-off, feature MD5s), loss curve and model artifacts.
Evaluation	NumPy, matplotlib	Ranking metrics at K=5 and 10 on warm test users, plus catalogue coverage, genre intra-list diversity, novelty and RMSE/MAE; metrics are written back to each MLflow run.

Stage	Tool(s)	Key design decision
Orchestration	Prefect 3 (+ plain-Python runner)	The DAG is declared once and used by both orchestrators; tasks have explicit dependencies, ingestion retries, and a cron schedule via `flow.serve`. The fallback runner skips downstream stages on failure.

3. Data validation and preparation results

Of 1,000,509 raw interactions, **520 were quarantined** (pass rate 99.948%) and 450 passed with warnings. All 6 of 6 injected error-type defect classes were detected in full. Users (43 warnings) and items (17) needed repair only, with no rejections.

Rule (interactions)	Severity	Rows	Rule (interactions)	Severity	Rows
duplicate_interaction	warning	304	missing_key	error	150
duplicate_row	warning	150	orphan_item	error	50
invalid_rating	error	120	orphan_user	error	50
invalid_timestamp	error	90	wrong_type	error	60

After cleaning, the time-based split at **2000-12-02** gives 799,725 training ratings (5,392 users, 3,662 items) and 103,783 warm test ratings from 1,135 users. 96,064 test ratings belong to users who joined after the cut-off. A personalised model cannot score these cold-start users, so they are reported and left out of the comparison; in serving they fall back to the popularity model.

4. Feature management

The registry defines **28 features** (14 user, 11 item and 3 on-demand user×item features; two of them are 19-dimensional genre vectors). All are computed *as of* the split cut-off (point-in-time correct), stored as parquet with MD5 checksums, and the checksums are logged to every MLflow training run. Examples: `user_avg_rating`, `user_activity_count`, `user_preferred_genre`, `user_days_since_last_interaction`, `item_popularity`, `item_bayes_rating`, `ui_genre_affinity`. The popularity and hybrid models read their signals from the store, and so does `src/serving/recommend.py`. The full catalog with types, sources and live statistics is in `reports/feature_catalog.md`.

5. Models and evaluation results

Popularity (baseline) recommends the most-rated unseen items. **Item-kNN** is item-based collaborative filtering with shrunk cosine similarity. **ALS-MF** is biased explicit matrix factorisation (48 factors, ALS-WR) that predicts ratings. **Hybrid** blends min-max-normalised item-kNN and ALS scores with the user×item genre affinity and item popularity features. Tuned values: item-kNN 300 neighbours with shrinkage 100; ALS lambda = 0.08; hybrid weights 0.45 / 0.25 / 0.05 / 0.25. Relevant = test rating of 4 or more; items already seen are excluded.

Model	P@10	R@10	NDCG@10	Hit@10	Cov@10	Div@10	RMSE	MAE
Popularity (baseline)	0.1986	0.0479	0.2129	0.6284	0.0374	0.7601	0.9595	0.7597
Item-kNN CF	0.1912	0.0548	0.2124	0.6221	0.1436	0.7370	-	-
ALS Matrix Factorisation	0.0954	0.0256	0.1006	0.4434	0.1671	0.7904	0.8941	0.6956
Hybrid (CF + MF + features)	0.2265	0.0646	0.2489	0.6732	0.1092	0.7443	-	-

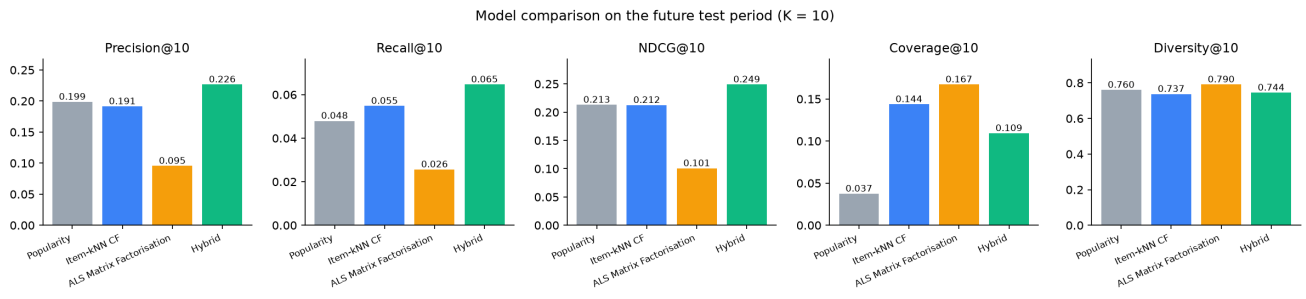


Figure 2 - Model comparison on the future test period (K = 10).

Interpretation. The **hybrid model is best on every accuracy metric**: NDCG@10 0.2489 vs 0.2129 (+16.9%), Recall@10 +35%, and about 2.9x the catalogue coverage. Popularity is a strong baseline under a temporal split, because the test period is dominated by mainstream titles. Item-kNN matches it on NDCG while covering about 4x

more items. ALS gives the best rating prediction (RMSE 0.894 vs 0.960 for the item-mean baseline), but ranking by predicted rating pushes long-tail items with high predicted scores (novelty 4.4), so its top-K accuracy is low. That is the usual gap between rating prediction and top-K ranking, and it is why ALS contributes to the hybrid as a signal and is not used on its own. Results are identical across repeated runs (fixed seed and deterministic splits): the MLflow runs table (A10) shows the same metrics for every execution.

6. Limitations and future work

• **Injected defects.** Data-quality issues are synthetic (seeded and logged), so the validation layer is proven against known defects, not against real production drift.

• **Cold start.** About half of the raw test ratings come from users who joined after the cut-off. They are served by popularity only. Demographic or content-based onboarding models could cover them.

• **Offline evaluation only.** A single temporal split on a 2000-2003 dataset. A rolling-origin backtest and online A/B metrics (CTR, watch time) would give a better estimate of real impact.

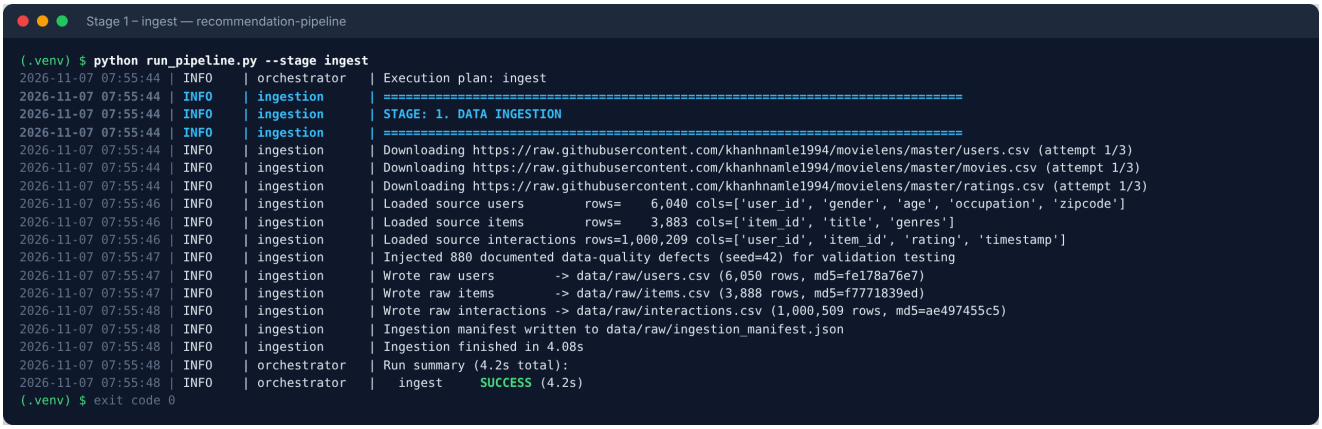
• **Scale.** The dense item-item similarity matrix and the pure-NumPy ALS loop suit 1M ratings. Larger catalogues need approximate nearest neighbours, the `implicit` library or Spark ALS.

• **Feature store.** The file-based store has no online, low-latency serving layer or TTLs. Feast with a Redis/Postgres online store is the natural next step. Hybrid tuning uses features built on the full train period, a mild optimistic bias limited to the choice of weights.

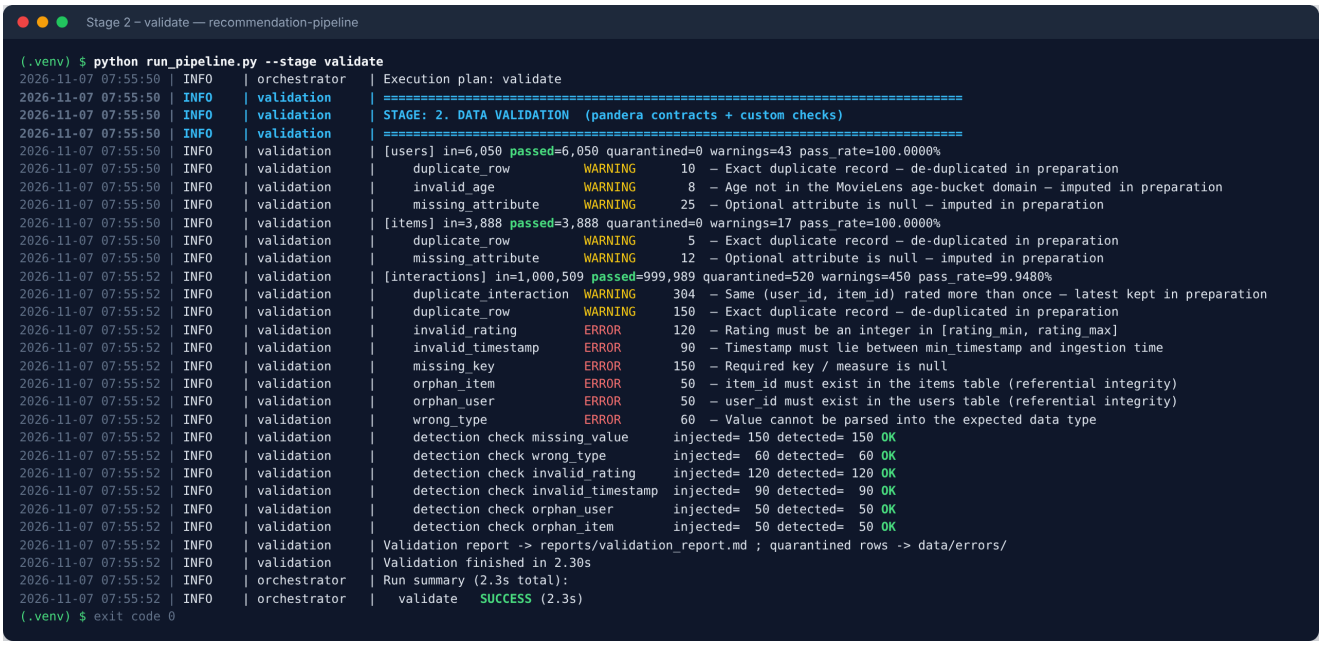
• **Model registry.** Models are tracked as MLflow artifacts, but they are not promoted through MLflow Model Registry stages, and the pipeline has no CI/CD.

Appendix A - Execution screenshots of every pipeline stage

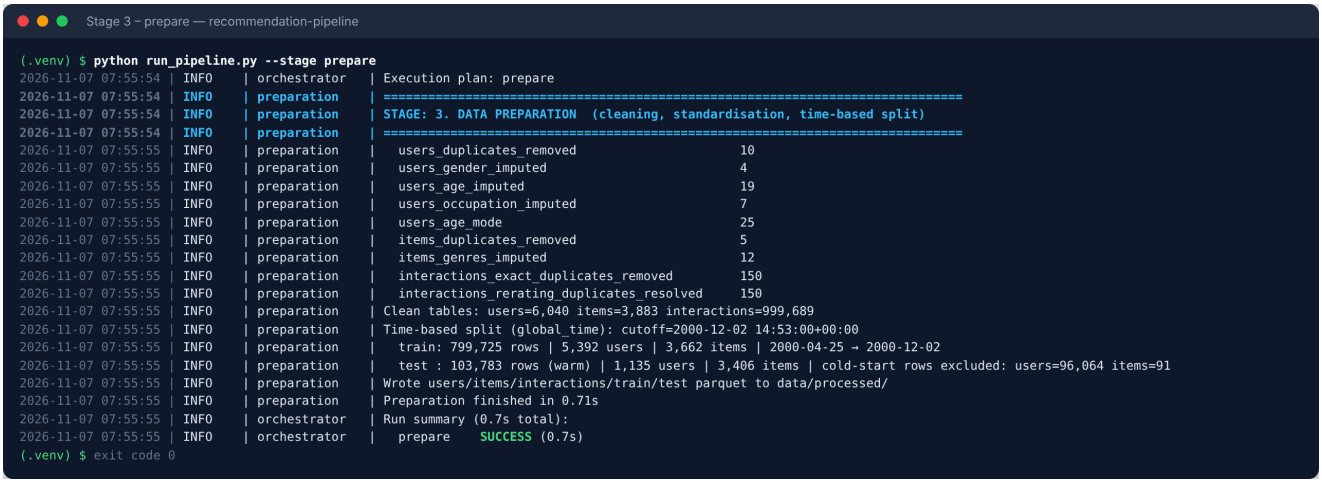
Each stage was executed individually with `python run_pipeline.py --stage <name>`, then the full DAG was executed as a Prefect flow against a local Prefect server. Terminal captures show the unedited program output (A8 shows only the Prefect orchestration lines; the complete flow output is in `reports/prefect_flow_run.log`). UI captures are from the Prefect and MLflow web UIs.



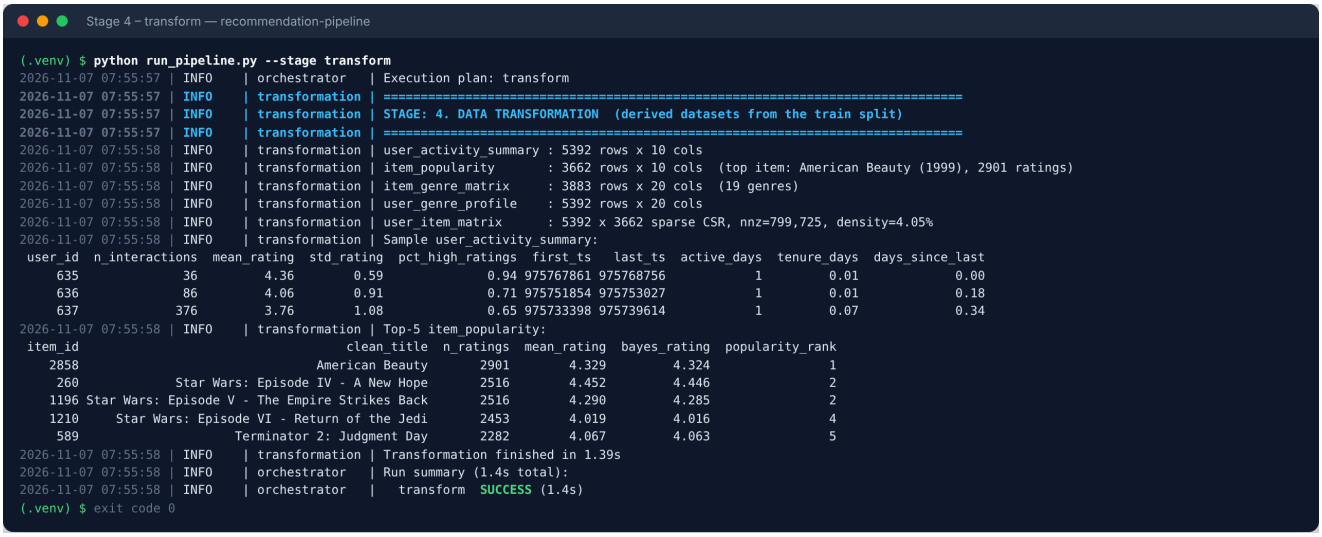
A1 - Stage 1, Data ingestion: download/cache, schema standardisation, defect injection, manifest with MD5.



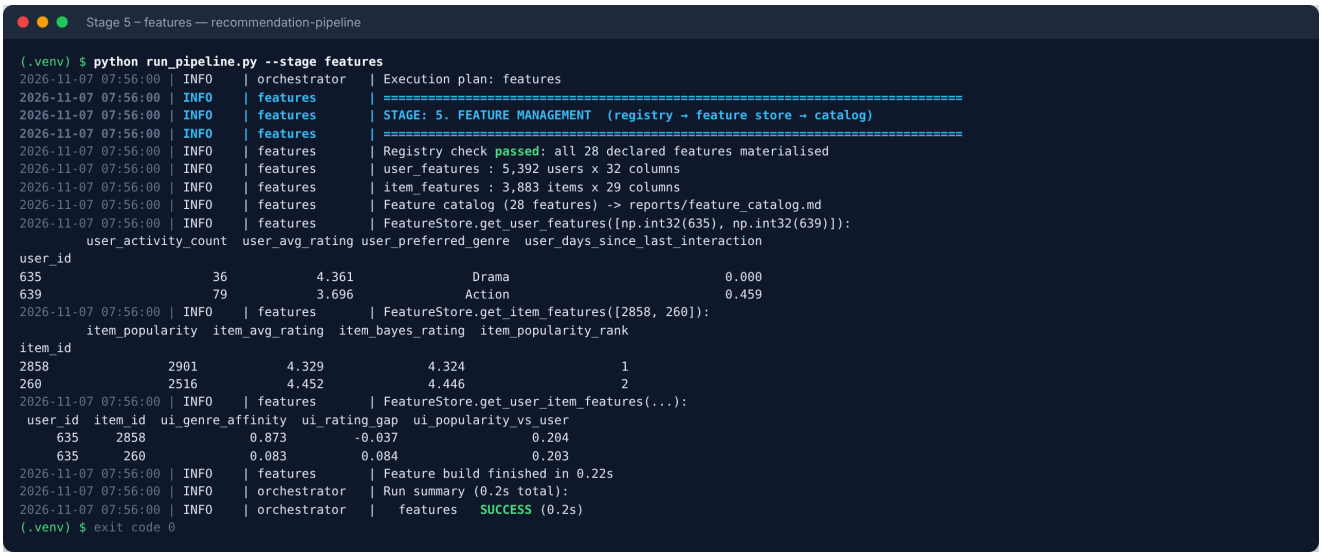
A2 - Stage 2, Data validation: pandera rule counts by severity, quarantine, detection check against injected defects.



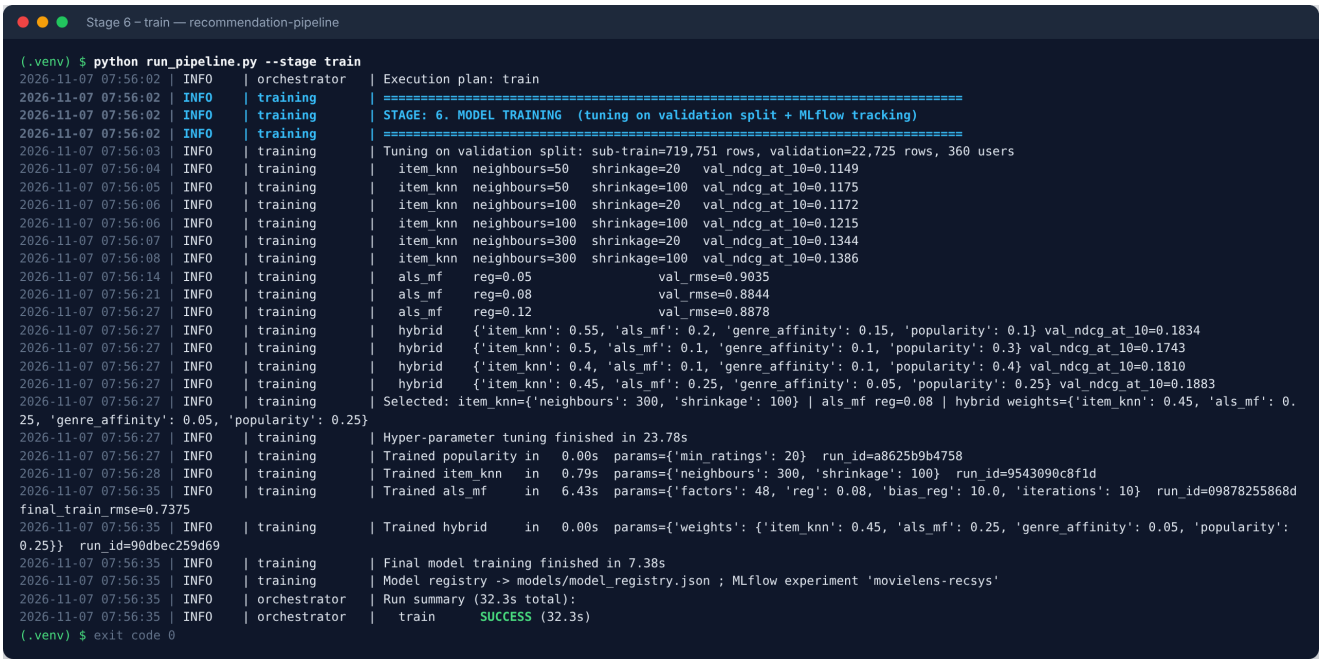
A3 - Stage 3, Data preparation: de-duplication and imputation counts, time-based train/test split.



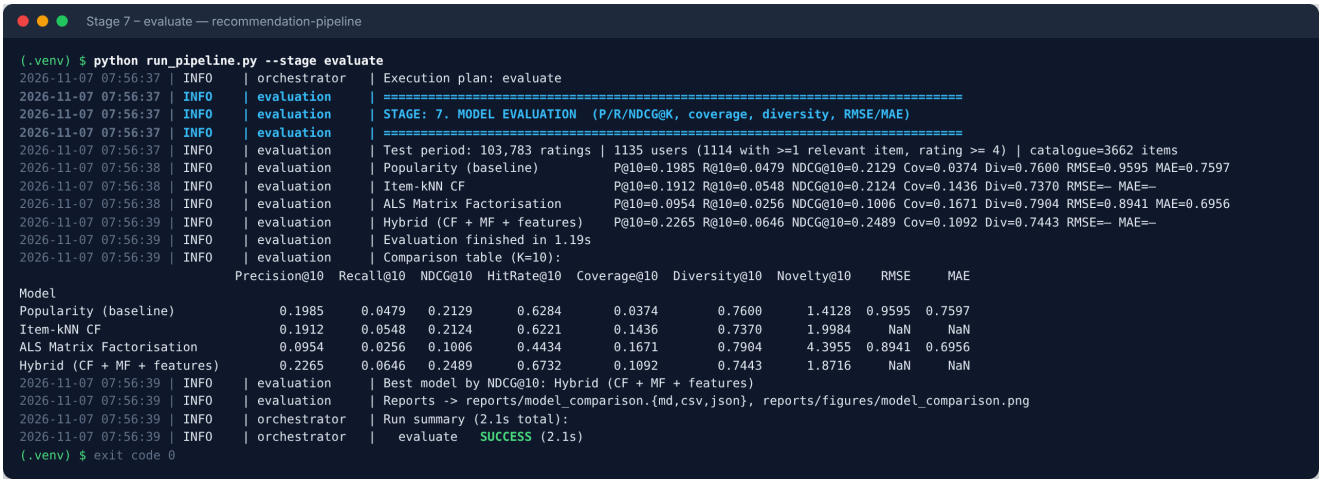
A4 - Stage 4, Data transformation: derived datasets (activity summary, item popularity, genre matrices, sparse user-item matrix).



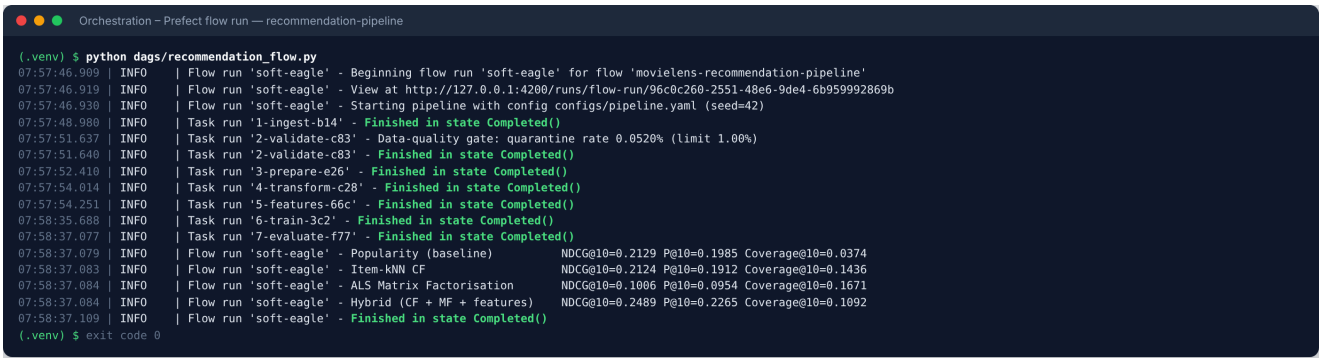
A5 - Stage 5, Feature management: registry check, feature store tables, catalog, FeatureStore API calls.



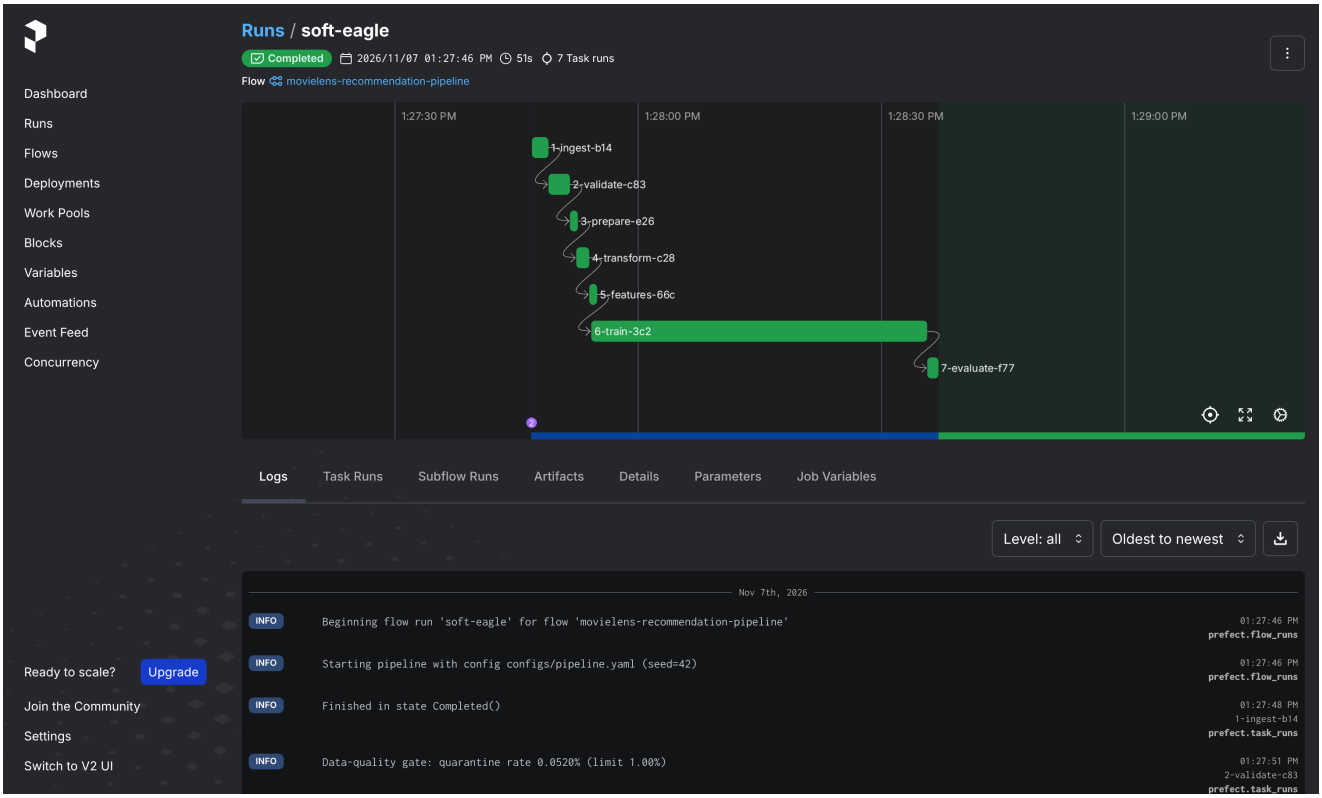
A6 - Stage 6, Model training: validation-split tuning (MLflow nested runs) and final training of the four models.



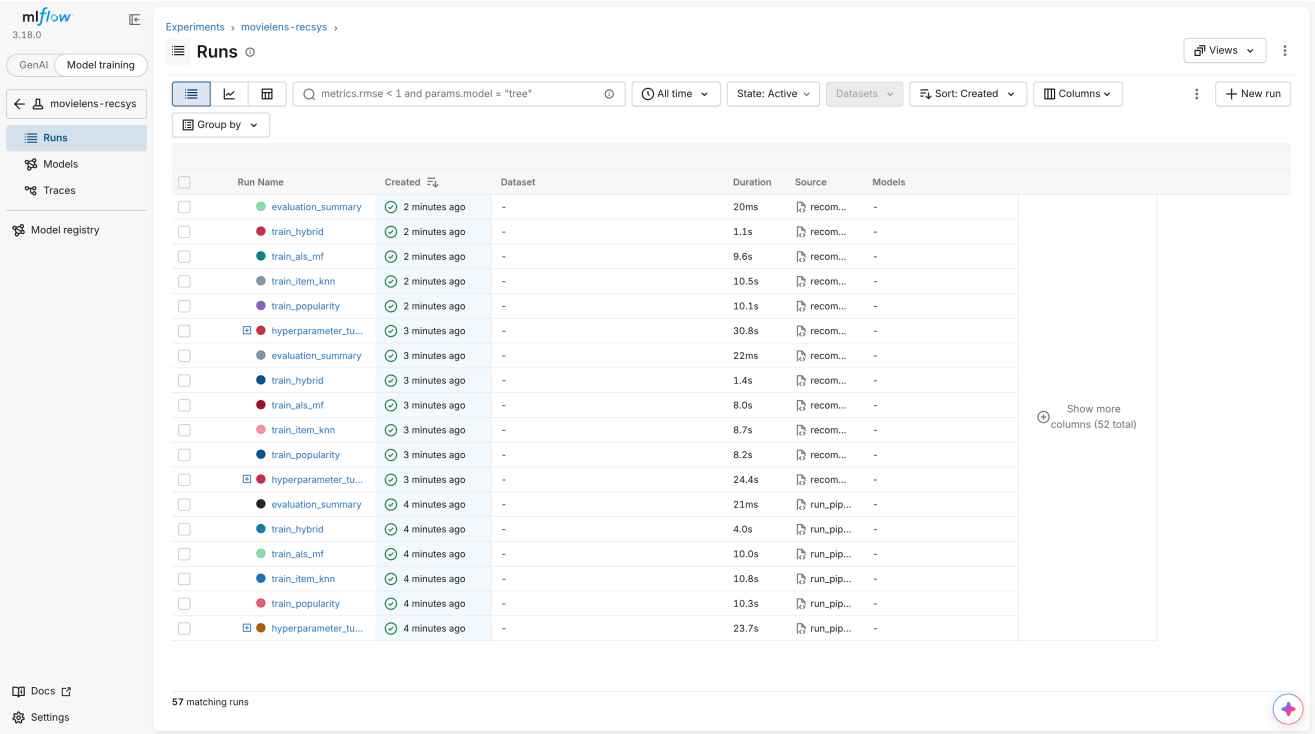
A7 - Stage 7, Model evaluation: P/R/NDCG@10, coverage, diversity, novelty, RMSE/MAE comparison.



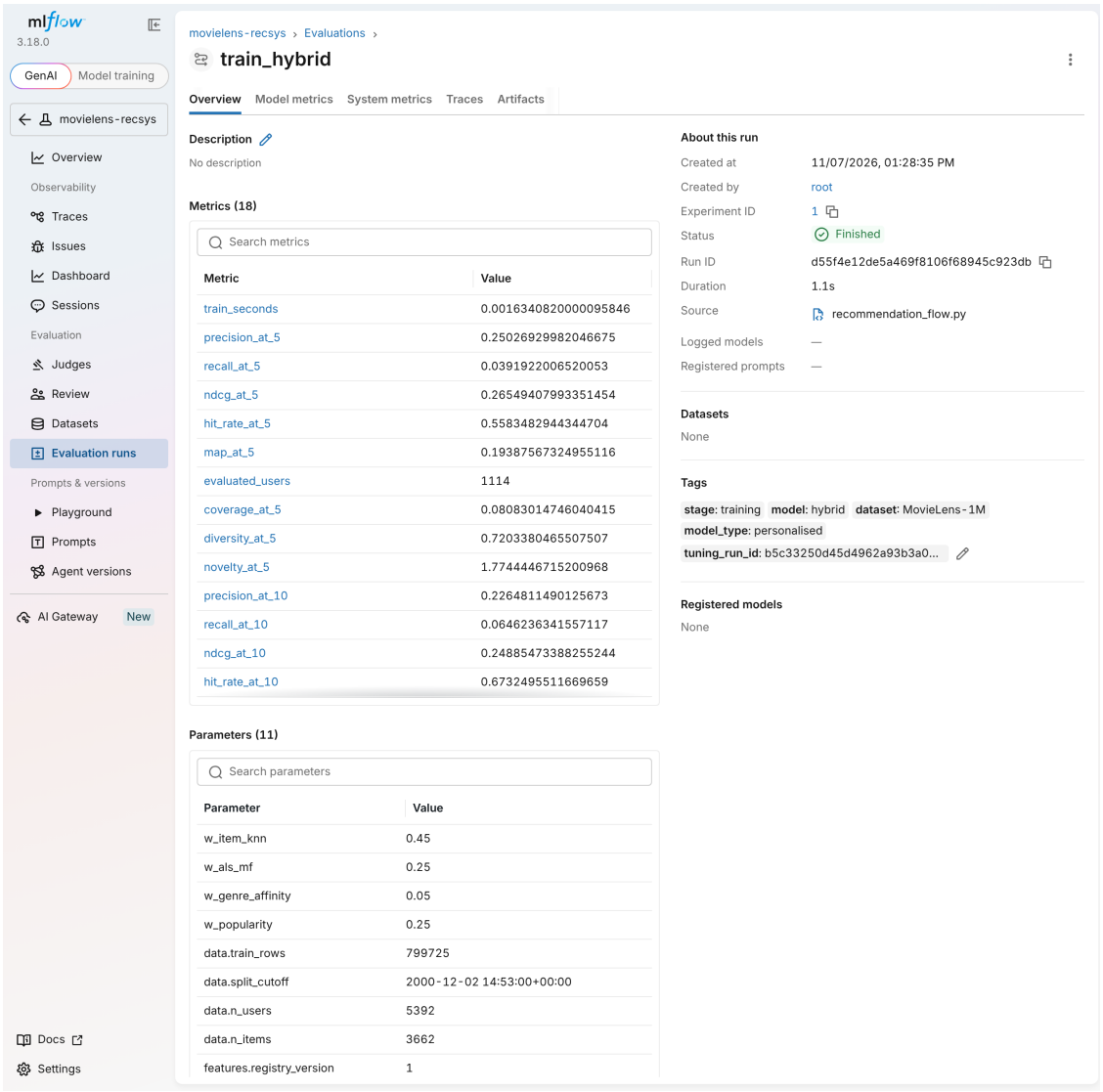
A8 - Orchestration: Prefect flow run summary - the seven tasks completed in dependency order, data-quality gate and final model scores.



A9 - Prefect UI: successful flow run, task dependency graph and logs (7/7 tasks Completed).



A10 - MLflow experiment 'movielens-recsys': tuning parent runs, training runs and evaluation summaries, one set per pipeline execution.



A11 - MLflow run detail (train_hybrid): logged metrics, parameters, lineage tags and link to the tuning run.

```
MLflow – tracked params & metrics — recommendation-pipeline

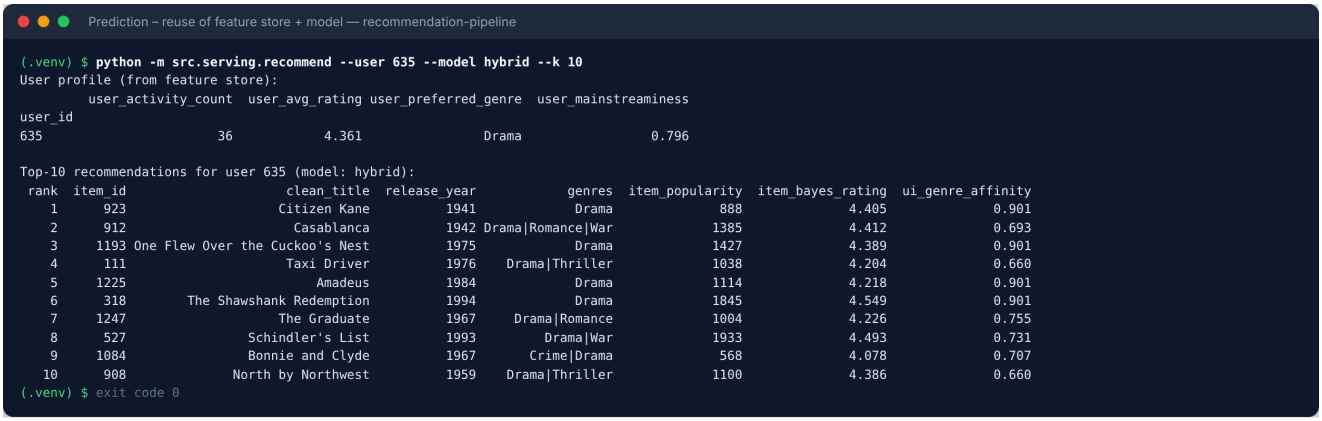
(.venv) $ python scripts/show_mlflow_runs.py
MLflow tracking URI: sqlite:////home/claude/recommendation-pipeline/mlflow.db
Experiment: movielens-recsys  |  latest pipeline execution (4 training runs)

Parameters:
      run_id      model neighbours shrinkage factors  reg min_ratings train_rows feat_ver
d55f4e12de5a     hybrid          –         –       –    –           –     799725        1
c12781616c6a     als_mf          –         –      48 0.08           –     799725        1
ffedc396240b   item_knn        300       100       –    –           –     799725        1
6cb6a392bc4d popularity          –         –       –    –          20     799725        1

Metrics:
     model   P@10   R@10  NDCG@10  Cov@10  Div@10   RMSE    MAE  train_s
    hybrid 0.2265 0.0646   0.2489  0.1092  0.7443      –      –   0.0016
    als_mf 0.0954 0.0256   0.1006  0.1671  0.7904 0.8941 0.6956   8.8867
  item_knn 0.1912 0.0548   0.2124  0.1436  0.7370      –      –   0.8878
popularity 0.1985 0.0479   0.2129  0.0374  0.7600 0.9595 0.7597   0.0022

Artifacts of run d55f4e12de5a: ['lineage', 'model']
(.venv) $ exit code 0
```

A12 - Tracked parameters and metrics of the latest execution, queried from MLflow.



A13 - Prediction: top-10 recommendations for user 635 from the hybrid model, with features read from the same feature store.



A14 - Unit tests for metrics and DAG ordering.